Social UNA: Plan de trabajo del equipo

Proyecto Final: Red Social Estudiantil · Estructuras de Datos · Universidad Nacional, Costa Rica
Integrantes: Kendall, Greivin y Mariano · Fecha de entrega y defensa: 10 de noviembre de 2026

Resumen del reparto

Persona	Punto del MVP	Archivo de datos propio
Kendall	1. Módulo de Autenticación (registro y login)	usuarios.json
Greivin	2. Módulo de Interacción Social (buscador, amigos)	amistades.json
Mariano	3. Módulo de Publicaciones (muro)	publicaciones.json

Cada persona desarrolla su módulo completo (ventanas, estructuras de datos, archivos y lógica). Así los cuatro criterios de la rúbrica (GUI 20%, Estructuras 30%, Persistencia 30%, Lógica 20%) quedan cubiertos por las tres.

0. Acuerdos del equipo (los tres, día 1)

1. Elegir el lenguaje y la librería de GUI (por ejemplo Java + Swing o Python + Tkinter). Todos deben poder ejecutarlo.
2. Crear un repositorio (GitHub) con una rama por persona.
3. Definir las clases del modelo, que no se cambian sin avisar: **Usuario** (id, nombre, usuario, contrasena), **Publicacion** (id, idAutor, texto, fechaHora) y **Amistad** (idUsuarioA, idUsuarioB).
4. Definir las estructuras en memoria: **ListaUsuarios** (lista dinámica, más un diccionario usuario → Usuario), **ListaPublicaciones** (lista dinámica) y **RedAmistades** (lista de adyacencia id → conjunto de ids, bidireccional).
5. Definir los archivos en la carpeta data/: usuarios.json, amistades.json y publicaciones.json. No se usa MySQL, SQL Server ni otra base de datos.
6. Dividir el código en carpetas: modelo/, datos/, servicios/ y gui/.
7. Crear una clase **Sesion** con el usuario activo y un **GestorArchivos** compartido con cargarTodo() y guardarTodo().
8. Definir el flujo de ventanas: Login ↔ Registro → Ventana principal con pestañas (Muro, Buscar, Amigos) y "Cerrar sesión".
9. Acordar el formato: ID único, usuario único y sin espacios, fecha en formato yyyy-MM-dd HH:mm:ss.
10. Revisar el foro del profesor al menos una vez al día: sus instrucciones cuentan como parte del enunciado.

1. Kendall: Módulo de Autenticación (punto 1)

Interfaz gráfica (20%)

1. Ventana de Login con usuario, contraseña (oculta) y botones "Ingresar" y "Crear cuenta".
2. Ventana de Registro con ID, Nombre, Usuario, Contraseña y confirmar contraseña, y botones "Registrar" y "Volver".
3. Validación de campos vacíos en ambas ventanas, con alerta gráfica.
4. Alertas para: usuario ya existe, ID repetido, credenciales incorrectas y registro exitoso.
5. Navegación fluida: login exitoso → ventana principal; registro exitoso → vuelve al login; "Cerrar sesión" → vuelve al login. Ninguna ventana debe bloquearse.

Estructuras de datos (30%)

1. Implementar ListaUsuarios con agregar, buscarPorUsuario, buscarPorId y existeUsuario.
2. Agregar el diccionario usuario → Usuario para que el login sea O(1).
3. Generar el ID siguiente automáticamente, o validar que no se repita.

Persistencia (30%)

1. Escribir guardarUsuarios() y cargarUsuarios() para usuarios.json.
2. Al iniciar el programa, leer el archivo y reconstruir la lista y el diccionario.
3. Guardar al registrar un usuario, al cerrar sesión y al cerrar la aplicación.
4. Manejar archivo inexistente o vacío sin que el programa falle.
5. Crear datos de prueba: al menos 3 usuarios pre-registrados (kendall, greivin, mariano).

Lógica de negocio (20%)

1. registrar(): campos no vacíos, usuario e ID únicos, contraseñas coincidentes.
2. login(): validar las credenciales leídas desde el archivo y guardar el usuario en Sesion.
3. Mensaje genérico si falla, sin revelar cuál campo estaba mal.
4. cerrarSesion(): limpiar Sesion y disparar el guardado.
5. Opcional: contraseña con hash (SHA-256).

Pruebas que debe hacer

Registrar un usuario, cerrar y reabrir el programa, y confirmar que el login funciona. También probar campos vacíos, usuario repetido y contraseña incorrecta.

2. Greivin: Módulo de Interacción Social (punto 2)

Interfaz gráfica (20%)

1. Pestaña "Buscar usuarios" con cuadro de texto, botón "Buscar" y tabla o lista de resultados.
2. Botón "Agregar como amigo" sobre el usuario seleccionado.
3. Pestaña "Mis amigos" con la lista de amigos del usuario activo.
4. Alertas para: búsqueda vacía, no seleccionó a nadie, ya son amigos, no puedes agregarte a ti mismo y amigo agregado.
5. La lista de amigos se refresca sin cerrar la ventana.
6. Opcional: botón "Eliminar amigo".

Estructuras de datos (30%)

1. Implementar RedAmistades (lista de adyacencia o grafo), con agregarAmistad(a, b) y sonAmigos(a, b).
2. Agregar obtenerAmigos(id), que devuelve objetos Usuario y no solo IDs.
3. La amistad es bidireccional: al agregar A–B se guarda en ambos sentidos.
4. Implementar la búsqueda sobre la lista de usuarios (coincidencia parcial, sin distinguir mayúsculas).

Persistencia (30%)

1. Guardar y cargar amistades.json (pares idA, idB).
2. Al iniciar, reconstruir el grafo en memoria desde el archivo.
3. Guardar inmediatamente después de cada amistad nueva, al cerrar sesión y al cerrar la aplicación.
4. Evitar duplicados (A–B y B–A son la misma amistad).
5. Crear datos de prueba: amistades entre los 3 usuarios pre-registrados.
6. Si un ID del archivo no existe en usuarios, ignorarlo sin fallar.

Lógica de negocio (20%)

1. buscarUsuarios(texto): excluye al usuario activo.
2. agregarAmigo(): valida que no sea uno mismo, que no sea duplicado y que el usuario exista.
3. Opcional: solicitud de amistad con aceptar o rechazar. Si no, basta con "Agregar como amigo", que el enunciado permite.
4. Exponer obtenerIdsVisibles(idUsuario) (el usuario y sus amigos). Mariano lo necesita para el muro.

Pruebas que debe hacer

Agregar un amigo, reabrir el programa y confirmar que sigue ahí. Verificar que el amigo también lo ve desde su cuenta.

3. Mariano: Módulo de Publicaciones y Muro (punto 3)

Interfaz gráfica (20%)

1. Pestaña "Muro" con un área de texto y el botón "Publicar".
2. Selector o pestañas para elegir **Muro general** (usuario + amigos) o **Mi muro** (solo mis publicaciones).
3. Lista de publicaciones con autor, fecha y texto, con barra de desplazamiento.
4. Validación de mensaje vacío con alerta, y opcionalmente un contador de caracteres.
5. El muro se refresca al publicar, sin cerrar la ventana.
6. Ventana principal con pestañas (Muro, Buscar, Amigos) y saludo con el nombre del usuario activo.
7. Cuidar el diseño (colores, fuentes, márgenes) para que sea atractivo e intuitivo.

Estructuras de datos (30%)

1. Implementar ListaPublicaciones con agregarPublicacion().
2. Implementar obtenerMuro(idUsuario): filtra por autor propio o amigo (usando el grafo de Greivin) y ordena de la más reciente a la más antigua.
3. Implementar obtenerMuroPersonal(idUsuario): solo las publicaciones del usuario.
4. Resolver el nombre del autor con el diccionario de Kendall.
5. Generar el ID de publicación de forma única.

Persistencia (30%)

1. Guardar y cargar publicaciones.json.
2. Al iniciar, reconstruir la lista en memoria.
3. Guardar al publicar, al cerrar sesión y al cerrar la aplicación.
4. Usar UTF-8 y manejar bien tildes, ñ, comillas y saltos de línea.
5. Crear datos de prueba: al menos 2 publicaciones por cada usuario pre-registrado, con fechas distintas.
6. Ignorar publicaciones con autor inexistente, sin fallar.

Lógica de negocio (20%)

1. publicar(): validar texto no vacío y asignar autor (usuario activo) y fecha/hora actual.
2. Visualización cruzada: si A y B son amigos, cada uno ve las publicaciones del otro. Si C no es amigo de ninguno, no las ve.
3. Orden cronológico inverso comprobado con las fechas de prueba.

Pruebas que debe hacer

Publicar con dos cuentas amigas y comprobar el orden. Luego iniciar sesión con un tercer usuario sin amistad y confirmar que no ve esas publicaciones.

4. Tareas compartidas (los tres)

Paso	Qué	Responsable
1	Reunión de acuerdos y repositorio	Los tres
2	Integrar los tres guardar/cargar en un solo GestorArchivos (cargarTodo al iniciar; guardarTodo al cerrar sesión y al cerrar la app)	Kendall
3	Integrar la ventana principal con pestañas y Sesion	Mariano
4	Archivos de datos de prueba finales (≥3 usuarios con publicaciones)	Cada uno aporta su archivo
5	Pruebas integradas con el checklist de la sección 5, probando cada uno el módulo de otro	Los tres
6	Empaquetar en un solo archivo comprimido: código fuente, ejecutable o instrucciones exactas, carpeta data/ y README	Greivin
7	UML opcional: diagrama de clases que explique cómo se estructuró la memoria y la lógica, más un manual técnico breve	Kendall (clases y memoria) y Mariano (lógica y flujo)
8	Preparar la defensa: cada quien explica su parte y todos conocen el flujo completo	Los tres

5. Checklist de integración por criterio de la rúbrica

Interfaz gráfica (20%)

[ ] No se usa la consola, solo ventanas, botones, cuadros de texto y alertas gráficas.
[ ] Todos los campos vacíos muestran alerta.
[ ] Se navega entre ventanas sin que se congelen.

Estructuras de datos (30%)

[ ] Usuarios y publicaciones están en listas dinámicas.
[ ] Las amistades están en un grafo o lista de adyacencia.
[ ] Durante la ejecución todo se consulta en memoria, sin releer el archivo.

Persistencia (30%)

[ ] Registrar, agregar amigo y publicar, cerrar sesión y cerrar el programa: nada se pierde al reabrir.
[ ] Los archivos de prueba cargan al iniciar (≥3 usuarios con publicaciones).
[ ] No hay base de datos relacional, solo archivos.
[ ] Un archivo ausente o vacío no rompe el programa.

Lógica de negocio (20%)

[ ] Login exitoso con credenciales del archivo y fallo con credenciales incorrectas.
[ ] La amistad se ve en ambos usuarios.
[ ] El muro muestra publicaciones propias y de amigos, de la más reciente a la más antigua.
[ ] Un usuario sin amistad no ve publicaciones ajenas.

Entregables

[ ] Código y ejecutable (o instrucciones exactas) comprimidos.
[ ] Archivos de datos de ejemplo.
[ ] UML y manual técnico (opcional).

6. Calendario

Fechas	Meta
2–3 nov	Acuerdos, repositorio, modelo y formato de archivos
3–6 nov	Cada uno desarrolla su parte en su rama
6–7 nov	Integración, GestorArchivos único y ventana principal
7–8 nov	Pruebas cruzadas, corrección de errores y datos de prueba
8–9 nov	Empaquetado, README, UML y ensayo de la defensa
10 nov	Entrega y defensa

7. Dependencias entre los tres

- Mariano necesita el grafo de Greivin y el diccionario de Kendall, así que las firmas de obtenerAmigos y buscarPorId se acuerdan el día 1.
- Greivin necesita ListaUsuarios de Kendall.
- Kendall depende menos de los demás, así que puede terminar primero y ayudar con la integración.
- Mientras el módulo de otro no esté listo, usar datos simulados para no bloquearse.